CLOCK / RESET GENERATION

ACLK
ARESETn

u_reset_sync_aclk
module reset_sync
clk
arst_n
rst_n_sync

u_sclk_gen
module sclk_gen
clk
rst_n
o_sclk

w_sclk_raw

u_bufg_sclk
module BUFG
I
O

SCLK

u_reset_sync_sclk
module reset_sync
clk
arst_n
rst_n_sync

sclk_rst_n

aclk_rst_n

o_dbg_sclk

SCLK

sclk_rst_n

EXTERNAL INTERFACE (spi_bridge_top) | MODULE HIERARCHY

in ACLK		spi_bridge_top
in ARESETn		├ u_reset_sync_aclk (reset_sync)
in i_bus_addr	[8]	├ u_reset_sync_sclk (reset_sync)
in i_bus_wr_en		├ u_sclk_gen (sclk_gen)
in i_bus_wdata	[8]	├ u_bufg_sclk (BUFG primitive)
in i_bus_rd_en		├ u_spi_master (spi_master)
out o_bus_rdata	[8]	├ u_spi_slave (spi_slave)
out o_overflow		├ u_rx_fifo (async_fifo 16×8)
out o_rx_valid		├ u_tx_fifo (async_fifo 4×8)
out o_dbg_sclk		├ u_overflow_pulse_sync (pulse_sync)
out o_dbg_cs_n		├ u_register_block (register_block)
out o_dbg_mosi		※ SPI 외부 포트·CPU·AXI 없음 (debug o_dbg_* 만)
out o_dbg_miso		

생략 신호 목록: 하단 OMITTED 표 / E-4

SCLK DOMAIN · 25 MHz

ACLK DOMAIN · 100 MHz

o_dbg_cs_n
o_dbg_mosi

SCLK
u_spi_master
module spi_master
1
SCLK
rst_n
state[1:0] FSM
idle_cnt/bit_cnt
r_tx_byte (+1)
r_shift_out
r_shift_in
IDLE_GAP=8
o_cs_n
o_mosi
i_miso
o_busy
o_rx_captured
o_rx_captured_valid
NC
NC

w_cs_n
w_mosi
w_miso
7
o_dbg_miso

SCLK
u_spi_slave
module spi_slave
2
SCLK
rst_n
i_cs_n
i_mosi
o_miso
o_rx_wr_en
o_rx_wr_data
i_rx_fifo_full

RX CAPTURE (posedge SCLK)
3
r_shift_rx <= {r_shift_rx[6:0],i_mosi}
bit_cnt[2:0]: 0..7 (MSB first)
w_byte_complete → S_BYTE_DONE
o_rx_wr_data = r_shift_rx

CONTROL FSM state[1:0]
S_IDLE → S_LOAD (!i_cs_n)
S_LOAD → S_RECEIVE
S_RECEIVE→ S_BYTE_DONE
(w_byte_complete)
S_BYTE_DONE → S_RECEIVE
i_cs_n=1 : 항상 S_IDLE 복귀

o_rx_overflow_pulse

TX HOLD / SHIFT
6
w_tx_pop=!empty&!hold_valid&!pop_d
r_tx_hold[7:0] / r_tx_hold_valid
r_tx_pop_d (FIFO 1-cycle 지연)
r_shift_tx <= r_tx_hold (진입 시)
o_miso <= r_shift_tx[7]
(negedge SCLK)

o_tx_fifo_rd_en
i_tx_fifo_rd_data
i_tx_fifo_empty
o_spi_busy

wr: SCLK
rd: ACLK
u_rx_fifo
module async_fifo #(DATA_WIDTH=8, ADDR_WIDTH=4)
wr_clk
wr_rst_n
i_wr_en
i_wr_data
o_full
rd_clk
rd_rst_n
i_rd_en
o_rd_data
o_empty
o_rd_level

mem[0:15] : 16×8
wr_clk 쓰기 / rd_clk 레지스터 읽기
2FF
5
wr_ptr_gray → 2FF → rd side
2FF
rd_ptr_gray → 2FF → wr side

WRITE SIDE
r_wr_ptr_bin
r_wr_ptr_gray
r_full

READ SIDE
r_rd_ptr_bin
r_rd_ptr_gray
r_empty

w_rx_wr_en
4
w_rx_wr_data[7:0]
w_rx_fifo_full

w_rx_fifo_rd_en
6
w_rx_fifo_rd_data[7:0]
w_rx_fifo_empty
w_rx_fifo_level[4:0]

w_rx_fifo_full

⚠ CDC-01 (SCLK→ACLK): 동기화 없이 register_block 에 직결

ACLK
u_register_block
module register_block
ACLK
rst_n
o_rx_fifo_rd_en
i_rx_fifo_rd_data
i_rx_fifo_empty
i_rx_fifo_level
i_rx_fifo_full
i_bus_addr
i_bus_wr_en
i_bus_wdata
i_bus_rd_en
o_bus_rdata
o_overflow
o_rx_valid

[7:0] i_bus_addr
1 i_bus_wr_en
[7:0] 2 i_bus_wdata
i_bus_rd_en
[7:0] 8 o_bus_rdata
o_overflow
o_rx_valid

ADDRESS MAP (i_bus_addr[7:0])
0x00 ADDR_RX_DATA RO r_rx_data_buf
0x04 ADDR_STATUS RO {000,OVF,RXV,BUSY,FULL,EMPTY}
0x08 ADDR_CONTROL RW [0]FIFO_CLEAR* [1]OVFL_CLEAR*
[2]irq_enable (*=write strobe)
0x0C ADDR_FIFO_LEVEL RO {000,i_rx_fifo_level[4:0]}
0x10 ADDR_TX_DATA WO→TX FIFO, read=last wdata
others → o_bus_rdata = 8'h00

REGISTERS / STATE (ACLK, sync reset)
7
r_rx_data_buf[7:0] + rx_data_buf_valid
r_rx_fifo_rd_en_d (prefetch 1-cycle 지연)
w_rx_prefetch=!valid&!empty&!drain
r_drain_active (FIFO_CLEAR → RX만 drain)
r_overflow_sticky ← pulse_sync, W1C
irq_enable (출력 없음)
o_tx_fifo_wr_en/_data (registered)

src: SCLK
dst: ACLK
u_overflow_pulse_sync
module pulse_sync
src_clk
src_rst_n
i_src_pulse
dst_clk
dst_rst_n
o_dst_pulse

toggle
meta
sync
sync_d
r_src_toggle
r_dst_meta→sync→sync_d
o_dst_pulse = r_dst_sync ^ r_dst_sync_d

w_rx_overflow_pulse
w_rx_overflow_pulse_sync

rd: SCLK
wr: ACLK
u_tx_fifo
module async_fifo #(DATA_WIDTH=8, ADDR_WIDTH=2)
rd_clk
rd_rst_n
i_rd_en
o_rd_data
o_empty
o_rd_level
wr_clk
wr_rst_n
i_wr_en
i_wr_data
o_full

mem[0:3] : 4×8
wr_clk 쓰기 / rd_clk 레지스터 읽기
4
2FF
wr_ptr_gray → 2FF → rd side
2FF
rd_ptr_gray → 2FF → wr side

READ SIDE
r_rd_ptr_bin
r_rd_ptr_gray
r_empty

WRITE SIDE
r_wr_ptr_bin
r_wr_ptr_gray
r_full

w_tx_fifo_rd_en
5
w_tx_fifo_rd_data[7:0]
w_tx_fifo_empty
NC

w_tx_fifo_wr_en
3
w_tx_fifo_wr_data[7:0]
w_tx_fifo_full

o_tx_fifo_wr_en
o_tx_fifo_wr_data
i_tx_fifo_full

i_spi_busy_sync

w_spi_busy_slave

w_spi_busy_master

SCLK

w_spi_busy

⚠ CDC-02 (SCLK→ACLK): OR 조합 출력이 2FF 입력으로 직결

r_spi_busy_meta
r_spi_busy_sync
D
Q
D
Q
r_spi_busy_sync
ACLK 2FF
RST_N
RST_N

(2FF level synchronizer in spi_bridge_top)

◀ CDC BOUNDARY ▶

SIGNAL FLOW — RX (실제 RTL 기준)

① u_spi_master → w_mosi/w_cs_n (negedge launch)
② u_spi_slave: posedge SCLK, r_shift_rx 시프트
③ bit_cnt=7 → S_BYTE_DONE (8-bit 완성)
④ o_rx_wr_en=!full → u_rx_fifo write (SCLK)
⑤ Gray wr ptr → 2FF → ACLK (CDC)
⑥ w_rx_prefetch → o_rx_fifo_rd_en (ACLK)
⑦ r_rx_data_buf / rx_data_buf_valid 적재
⑧ 버스 read 0x00 → o_bus_rdata (valid 소비)

SIGNAL FLOW — TX (실제 RTL 기준)

① i_bus_wr_en && addr==0x10 && !i_tx_fifo_full
② o_tx_fifo_wr_data <= i_bus_wdata (registered)
③ u_tx_fifo write (ACLK)
④ Gray wr ptr → 2FF → SCLK (CDC)
⑤ slave w_tx_pop → r_tx_hold (rd_en)
⑥ r_shift_tx → o_miso (negedge SCLK)
⑦ w_miso → master i_miso (캡처값 NC)

SIGNAL LEGEND

RX data (SPI→RX FIFO→reg)
TX data (reg→TX FIFO→MISO)
Clock
Reset
Control / Handshake (en, cs_n)
Status / Flag (full, empty, level, busy, overflow)
Register bus (i_bus_*, o_bus_rdata)
CDC path (domain crossing)
● junction ✕ NC(미연결) ▷ clock pin / 숫자 = bit width

OMITTED / NOT DRAWN (도면에서 생략하거나 기호로 대체한 항목 — RTL 에는 존재)

① 서브모듈 내부 net 은 Sheet B 에서 표현: sclk_gen r_toggle_phase · reset_sync r_meta_ff · async_fifo w_*_next, gray2bin() · spi_master idle_cnt/setup_cnt · register_block w_*_hit / w_*_strobe / w_status_reg_val / w_control_reg_val
② 파라미터: spi_master IDLE_GAP=8, async_fifo DATA_WIDTH/ADDR_WIDTH (u_rx_fifo 8/4, u_tx_fifo 8/2) — 헤더 문자열로만 표기
③ 미연결(NC, ✕ 표시): u_spi_master.o_rx_captured[7:0], u_spi_master.o_rx_captured_valid, u_tx_fifo.o_rd_level[2:0]
④ top 의 assign: o_dbg_sclk=SCLK, o_dbg_cs_n=w_cs_n, o_dbg_mosi=w_mosi, o_dbg_miso=w_miso → 출력 박스(tap)로 표현 / w_spi_busy = w_spi_busy_master || w_spi_busy_slave → OR 기호
⑤ r_spi_busy_meta / r_spi_busy_sync 는 top 내부 reg (인스턴스 아님) → DFF 기호. 전체 연결 목록은 Sheet E(연결표) 참조

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC

SHEET 1/9

FULL RTL ARCHITECTURE — TOP LEVEL

spi_bridge_top · 10 instances · 2 clock domains · A3 landscape

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…)

DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

u_spi_master (module spi_master)

SCLK domain · async rst_n (= sclk_rst_n) · parameter IDLE_GAP = 8

모든 FF: posedge SCLK / async-assert rst_n (active-low). 예외: o_mosi = negedge SCLK. SPI Mode 0, MSB-first.

SCLK (모든 블록 CK)

rst_n (모든 블록 async reset)

state → 카운터 조건 디코드 (S_IDLE / S_CS_SETUP / S_SHIFT)

idle_cnt[7:0]

```
S_IDLE : idle_cnt <= idle_cnt+1
그 외 : idle_cnt <= 0
w_idle_gap_done =
(idle_cnt == IDLE_GAP-1)
```

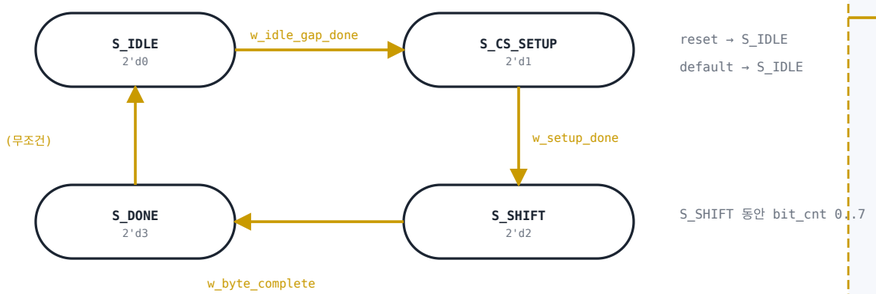
setup_cnt (1 bit)

```
S_CS_SETUP : setup_cnt <= 1
그 외 : setup_cnt <= 0
w_setup_done = (setup_cnt == 1)
```

bit_cnt[2:0]

```
S_SHIFT : bit_cnt <= bit_cnt+1
그 외 : bit_cnt <= 0
w_byte_complete =
(S_SHIFT && bit_cnt == 7)
```

FSM : state[1:0] (state reg / next-state 조합 / output-datapath 3-process)



state[1:0] (reg)

```
@(posedge SCLK or negedge rst_n)
reset : state <= S_IDLE
else : state <= next_state
```

next_state (always @(*))

```
next_state = state; // 기본값
case(state)
S_IDLE : gap_done → S_CS_SETUP
S_CS_SETUP: setup_done→ S_SHIFT
S_SHIFT : byte_done → S_DONE
S_DONE : → S_IDLE
default : → S_IDLE
```

MASTER 동작 순서 (RTL 구조에서 도출 · cycle 수는 정독 기준, 파형 재확인 필요)

1) S_IDLE : idle_cnt 0..IDLE_GAP-1 (8 SCLK cycle) 동안 o_cs_n=1
gap 완료 cycle: o_cs_n<=0, r_shift_out<=r_tx_byte, state→S_CS_SETUP
2) S_CS_SETUP : setup_cnt 0→1 (2 cycle) 후 S_SHIFT
3) S_SHIFT : bit_cnt 0..7 (8 cycle). o_mosi 는 negedge SCLK 에서 갱신,
slave 는 다음 posedge 에서 샘플 (Mode 0)
4) S_DONE : 1 cycle. o_cs_n<=1, r_tx_byte+1, o_rx_captured<=r_shift_in
5) 캡처값 o_rx_captured*, 및 i_miso 입력은 내부에서만 쓰이고
spi_bridge_top 은 o_rx_captured* 를 사용하지 않음 (NC)

r_tx_byte[7:0]

```
S_DONE : r_tx_byte <= r_tx_byte + 1
reset : 8'd0 (프레임마다 +1 패턴)
```

r_tx_byte

r_shift_out[7:0]

```
load : S_IDLE && w_idle_gap_done
r_shift_out <= r_tx_byte
S_SHIFT: {r_shift_out[6:0], 1'b0}
(shift left, MSB-first)
```

r_shift_out[7]

o_mosi (reg)

```
@(negedge SCLK or negedge rst_n)
S_SHIFT : o_mosi <= r_shift_out[7]
```

o_mosi → w_mosi → **o_mosi**

o_cs_n (reg)

```
reset : o_cs_n <= 1
S_IDLE && w_idle_gap_done : 1 → 0
S_DONE : 0 → 1
```

o_cs_n → w_cs_n → **o_cs_n**

i_miso ← i_miso ← w_miso (u_spi_slave.o_miso)

r_shift_in[7:0]

```
@(posedge SCLK) S_SHIFT :
r_shift_in <= {r_shift_in[6:0], i_miso}
(MSB-first 캡처, Mode 0 : posedge 샘플)
```

o_rx_captured[7:0]

```
S_DONE : <= r_shift_in
(debug 용 출력)
```

o_rx_captured[7:0] → **o_rx_captured**

NC — spi_bridge_top 에서 연결 안 됨

o_rx_captured_valid

```
S_DONE : 1-cycle pulse
```

o_rx_captured_valid → **o_rx_captured_valid**

NC — spi_bridge_top 에서 연결 안 됨

o_busy

```
assign o_busy = (state != S_IDLE)
```

o_busy → w_spi_busy_master (→ OR → 2FF, Sheet A·C 의 CDC-02) → **o_busy**

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC — SHEET 2/9

MODULE DETAIL — SPI MASTER

u_spi_master (module spi_master) · SCLK domain

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…) — DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

u_spi_slave (module spi_slave)

SCLK domain · async rst_n (= sclk_rst_n) · FIFO 인터페이스 = SCLK 쪽

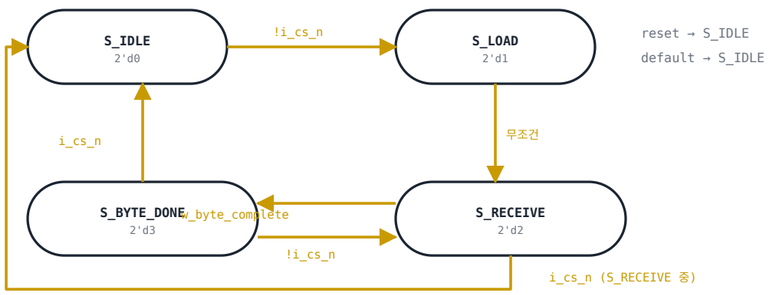
모든 FF: posedge SCLK / async-assert rst_n (active-low). 예외: o_miso = negedge SCLK. SPI Mode 0, MSB-first.

SCLK (모든 블록 CK)

rst_n (모든 블록 async reset)

i_cs_n

FSM : state[1:0] (state reg / next-state 조합 / datapath)



reset → S_IDLE

default → S_IDLE

state[1:0] (reg)

```
@(posedge SCLK or negedge rst_n)
reset: S_IDLE / else next_state
```

SLAVE 동작 요약 (RTL 정독 기준)

· i_cs_n=0 → S_IDLE→S_LOAD→S_RECEIVE. S_RECEIVE 에서 posedge SCLK 마다 i_mosi 를 r_shift_rx 로 시프트, bit_cnt==7 이면 S_BYTE_DONE (1 cycle)
· BYTE_DONE: full 이 아니면 o_rx_wr_en=1 (쓰기), full 이면 o_rx_overflow_pulse=1 (바이트 폐기, 펄스 → pulse_sync)
· TX: FIFO 가 비어있지 않고 hold 가 비어 있으면 w_tx_pop. FIFO read 는 registered(1-cycle) → r_tx_pop_d 로 정렬 후 hold 에 적재 (RTL-006 수정 구조)
· byte 경계(w_entering_receive)에서 hold 가 유효하면 r_shift_tx 로 로드, 아니면 8'h00 송출. o_miso 는 negedge SCLK 에서 갱신 (Mode 0)
· TX 데이터 없음(underflow)을 알리는 신호는 RTL 에 없음 (확인!)

← w_mosi

i_mosi

r_shift_rx[7:0]

```
@(posedge SCLK) S_RECEIVE :
r_shift_rx <= {r_shift_rx[6:0], i_mosi}
reset / entering_receive 외 유지
```

o_rx_wr_data[7:0] = r_shift_rx → **o_rx_wr_data**

state == S_BYTE_DONE

(디코드, 1 cycle)

RX FIFO write control

```
o_rx_wr_en = (state==S_BYTE_DONE) && !i_rx_fifo_full
o_rx_overflow_pulse =
(state==S_BYTE_DONE) && i_rx_fifo_full
(full 이면 바이트 폐기 + 펄스)
```

→ **o_rx_wr_en**

→ **o_rx_overflow_pulse**

i_rx_fifo_full ← w_rx_fifo_full (u_rx_fifo.o_full, SCLK 도메인)

w_byte_complete

bit_cnt[2:0]

```
S_RECEIVE : bit_cnt <= bit_cnt+1
w_entering_receive : bit_cnt <= 0
w_byte_complete =
(S_RECEIVE && bit_cnt == 7)
```

o_tx_fifo_rd_en = w_tx_pop → **o_tx_fifo_rd_en**

w_tx_pop (assign)

```
= !i_tx_fifo_empty
&& !r_tx_hold_valid
&& !r_tx_pop_d
```

i_tx_fifo_empty

r_tx_pop_d (FF)

```
r_tx_pop_d <= w_tx_pop
(FIFO 1-cycle read 지연 정렬)
```

r_tx_hold[7:0] / r_tx_hold_valid

```
r_tx_pop_d : hold <= i_tx_fifo_rd_data
hold_valid <= 1
w_tx_consume : hold_valid <= 0
```

w_tx_consume (assign)

```
= (state == S_RECEIVE)
&& (bit_cnt == 0)
&& !i_cs_n
&& r_tx_cur_from_hold
```

i_tx_fifo_rd_data ← w_tx_fifo_rd_data (u_tx_fifo.o_rd_data) [7:0]

r_tx_hold

o_spi_busy

```
assign o_spi_busy = (state != S_IDLE)
```

r_shift_tx[7:0] / r_tx_cur_from_hold

```
w_entering_receive :
hold_valid ? r_tx_hold : 8'h00
r_tx_cur_from_hold <= hold_valid
S_RECEIVE : {r_shift_tx[6:0], 1'b0}
```

[7] → **o_miso (reg)**

```
@(negedge SCLK)
S_RECEIVE: <= r_shift_tx[7]
```

→ **o_miso**

→ w_miso → u_spi_master.i_miso

o_spi_busy → w_spi_busy_slave (→ OR → 2FF, CDC-02) → **o_spi_busy**

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC SHEET 3/9

MODULE DETAIL — SPI SLAVE

u_spi_slave (module spi_slave) · SCLK domain

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…) DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

async_fifo #(DATA_WIDTH, ADDR_WIDTH) — u_rx_fifo / u_tx_fifo 공통 구조

Gray-pointer asynchronous FIFO · registered full/empty · registered read data

AW = ADDR_WIDTH. 포인터 폭 = AW+1 (wrap 비트 포함). DEPTH = 2^AW. wr_rst_n / rd_rst_n 은 각 도메인에서 async-assert(negedge) 로 사용.

WRITE DOMAIN (wr_clk)

wr_clk

wr_rst_n

i_wr_data

i_wr_data[DW-1:0]

i_wr_en

write-pointer next (always @(*))

```
w_wr_ptr_bin_next = r_wr_ptr_bin
+ (i_wr_en && !o_full)
w_wr_ptr_gray_next = (bin_next>>1)^bin_next
w_we = i_wr_en && !o_full (mem 쓰기 허가)
```

w_we

*_next

r_wr_ptr_bin[AW:0] / r_wr_ptr_gray[AW:0]

```
@(posedge wr_clk or negedge wr_rst_n)
reset: 0 else: <= *_next
```

wr addr

w_wr_ptr_gray_next

r_full (reg → o_full)

```
r_full <=
(w_wr_ptr_gray_next ==
{~r_rd_ptr_gray_wclk[AW:AW-1],
r_rd_ptr_gray_wclk[AW-2:0]})
reset: 0
```

o_full

o_full

r_rd_ptr_gray_wclk

rd ptr → wr domain (2FF)

```
r_rd_ptr_gray_meta_w <= r_rd_ptr_gray
r_rd_ptr_gray_wclk <= _meta_w
(@posedge wr_clk, async reset 0)
```

mem[0:DEPTH-1] (reg [DW-1:0])

```
쓰기 포트 : @(posedge wr_clk)
if (wr_rst_n && i_wr_en && !o_full)
mem[r_wr_ptr_bin[AW-1:0]] <= i_wr_data

읽기 포트 : @(posedge rd_clk)
if (rd_rst_n && i_rd_en && !o_empty)
o_rd_data <= mem[r_rd_ptr_bin[AW-1:0]]

→ 읽기 데이터는 1 cycle 뒤 o_rd_data 에서 유효 (registered read)
```

RTL 관찰 (정독 기준)

- full : write 포인터 next Gray 가 동기화된 read Gray 의 상위 2비트 반전과 같을 때 (registered)
- empty: read 포인터 next Gray 가 동기화된 write Gray 와 같을 때 (registered)
- 읽기 데이터는 registered → consumer 는 1-cycle 지연 처리 필요
(RTL-008: register_block RX prefetch 가 이를 일부 놓침 — 미수정)
- i_wr_en/i_rd_en 은 full/empty 일 때 내부에서 무시됨 (&& !o_full / !o_empty)

r_wr_ptr_gray[AW:0] (Gray, 1-bit 변화)

r_rd_ptr_gray[AW:0] (Gray, 1-bit 변화)

포인터는 Gray 코드로 한 비트씩만 변하며, 각 도메인에서 2FF 로 동기화됨 (데이터 버스는 동기화하지 않음)

full/empty 는 보수적: 동기화 지연으로 pessimistic (실제보다 늦게 해제)

READ DOMAIN (rd_clk)

rd_clk

rd_rst_n

i_rd_en

read-pointer next (always @(*))

```
w_rd_ptr_bin_next = r_rd_ptr_bin
+ (i_rd_en && !o_empty)
w_rd_ptr_gray_next = (bin_next>>1)^bin_next
```

*_next

r_rd_ptr_bin[AW:0] / r_rd_ptr_gray[AW:0]

```
@(posedge rd_clk or negedge rd_rst_n)
reset: 0 else: <= *_next
```

rd addr

gray_next

r_empty (reg → o_empty)

```
r_empty <=
(w_rd_ptr_gray_next ==
r_wr_ptr_gray_rclk)
reset: 1
```

o_empty

o_empty

r_wr_ptr_gray_rclk

wr ptr → rd domain (2FF)

```
r_wr_ptr_gray_meta_r <= r_wr_ptr_gray
r_wr_ptr_gray_rclk <= _meta_r
(@posedge rd_clk, async reset 0)
```

mem read

read data / level

```
o_rd_data : mem[r_rd_ptr_bin[AW-1:0]]
(i_rd_en && !o_empty 일 때 래치)
o_rd_level = gray2bin(r_wr_ptr_gray_rclk)
- r_rd_ptr_bin [AW:0]
(read 도메인, 동기화된 write 포인터 기준)
```

o_rd_data

o_rd_level

◀ CDC ▶

INSTANCES in spi_bridge_top

instance	DW	AW	DEPTH	ptr 폭	wr_clk / wr_rst_n	rd_clk / rd_rst_n	o_rd_level
u_rx_fifo	8	4	16	5	SCLK / sclk_rst_n	ACLK / aclk_rst_n	[4:0] → register_block.i_rx_fifo_level
u_tx_fifo	8	2	4	3	ACLK / aclk_rst_n	SCLK / sclk_rst_n	[2:0] 미연결 (NC)

방향: u_rx_fifo = SCLK 쓰기 → ACLK 읽기 (RX). u_tx_fifo = ACLK 쓰기 → SCLK 읽기 (TX). 같은 RTL 모듈의 두 인스턴스.

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC

SHEET 4/9

MODULE DETAIL — ASYNC FIFO

async_fifo (u_rx_fifo AW=4 / u_tx_fifo AW=2) · Gray-pointer CDC

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…)

DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

u_register_block (module register_block)

ACLK domain · synchronous reset rst_n (= aclk_rst_n) · 5 registers, 1 read mux

모든 FF: @(posedge ACLK), sync reset (if (!rst_n)). 같은 이름의 flag = 같은 net. 점선 flag = 모듈 내부 net (다른 블록에서 연결).

ACLK

rst_n

(모든 블록 CK / sync reset)

(decode strobes)

i_bus_addr [7:0]

i_bus_wr_en

i_bus_wdata [7:0]

ADDRESS DECODE (assign)

```
w_rx_addr_hit = (i_bus_addr == 8'h00)
w_tx_addr_hit = (i_bus_addr == 8'h10)
w_control_wr_hit = i_bus_wr_en && (i_bus_addr == 8'h08)
w_fifo_clear_strobe = w_control_wr_hit && i_bus_wdata[0]
w_ovfl_clear_strobe = w_control_wr_hit && i_bus_wdata[1]
```

i_bus_rd_en

i_rx_fifo_rd_data [7:0]

i_rx_fifo_empty

RX PREFETCH BUFFER

```
w_rx_prefetch = !rx_data_buf_valid
&& !i_rx_fifo_empty && !r_drain_active
o_rx_fifo_rd_en = w_rx_prefetch || r_drain_active
r_rx_fifo_rd_en_d <= w_rx_prefetch (read 1-cycle 정렬)
우선순위(posedge ACLK):
r_drain_active → valid<=0 (폐기)
r_rx_fifo_rd_en_d → buf<=i_rx_fifo_rd_data, valid<=1
i_bus_rd_en && w_rx_addr_hit && valid → valid<=0
```

⚠ w_rx_prefetch 에 !r_rx_fifo_rd_en_d 없음 → RTL-008 (미수정)

o_rx_fifo_rd_en = w_rx_prefetch || r_drain_active → o_rx_fifo_rd_en

o_rx_valid = rx_data_buf_valid → o_rx_valid

i_rx_fifo_empty

r_drain_active (FF)

```
set : w_fifo_clear_strobe
clr : r_drain_active && i_rx_fifo_empty
(RX FIFO 만 drain — TX FIFO 비움 없음)
```

irq_enable (FF)

```
w_control_wr_hit : irq_enable <= i_bus_wdata[2]
```

⚠ 출력 포트·소비처 없음 (읽기 값만 CONTROL[2])

i_rx_overflow_pulse_sync

r_overflow_sticky (FF)

```
i_rx_overflow_pulse_sync → 1 (set 우선)
w_ovfl_clear_strobe → 0
o_overflow = r_overflow_sticky
```

o_overflow → o_overflow

i_tx_fifo_full

TX WRITE REGISTERS (FF)

```
o_tx_fifo_wr_en <= i_bus_wr_en && w_tx_addr_hit
&& !i_tx_fifo_full
o_tx_fifo_wr_data <= i_bus_wdata (wr_en && tx hit)
```

o_tx_fifo_wr_en → o_tx_fifo_wr_en

o_tx_fifo_wr_data[7:0] → o_tx_fifo_wr_data

관찰 사항 (RTL 정독 기준)

- TX FIFO 가 full 이면 쓰기는 조용히 폐기됨 (o_tx_fifo_wr_en 만 0) — 오류 플래그 없음
- FIFO_CLEAR 는 RX FIFO 만 비움 (r_drain_active). irq_enable 은 소비처 없음
- RX_DATA 읽기 값은 valid 가 아니어도 r_rx_data_buf 현재값이 반환됨
- i_rx_fifo_full 은 SCLK 도메인 신호가 동기화 없이 STATUS[1] 로 들어감 (CDC-01)

STATUS w_status_reg_val[7:0]

i_rx_fifo_empty
r_overflow_sticky
rx_data_buf_valid
i_rx_fifo_full
i_spi_busy_sync

```
[7:5] = 3'b000 (reserved)
[4] OVERFLOW = r_overflow_sticky
[3] RX_VALID = rx_data_buf_valid
[2] SPI_BUSY = i_spi_busy_sync
[1] FULL = i_rx_fifo_full
[0] EMPTY = i_rx_fifo_empty
```

CONTROL w_control_reg_val[7:0]

irq_enable

```
{5'b00000, irq_enable, 1'b0, 1'b0}
[2] irq_enable (RW) · [1:0] 읽으면 0 (쓰기 strobe)
```

REGISTER MAP (i_bus_addr[7:0] — 8-bit 주소, 4-byte 간격)

주소	이름 (localparam)	접근	비트 필드 (RTL)
0x00	ADDR_RX_DATA	RO	[7:0] r_rx_data_buf (읽으면 valid 소비)
0x04	ADDR_STATUS	RO	[4]OVF [3]RXV [2]BUSY [1]FULL [0]EMPTY
0x08	ADDR_CONTROL	RW	[2]irq_enable [1]OVFL_CLEAR* [0]FIFO_CLEAR* (*strobe)
0x0C	ADDR_FIFO_LEVEL	RO	[4:0] i_rx_fifo_level ([7:5]=0)
0x10	ADDR_TX_DATA	WO*	[7:0] 쓰기→TX FIFO / 읽기=마지막 쓴 값

그 외 주소 → 0x00 반환 (RTL 기준; 참조 이미지의 주소와 다름).

(mux 입력)

r_rx_data_buf
w_status_reg_val
w_control_reg_val
i_rx_fifo_level
o_tx_fifo_wr_data
i_bus_addr

READ MUX (always @(*), case(i_bus_addr))

```
8'h00 : o_bus_rdata = r_rx_data_buf
8'h04 : o_bus_rdata = w_status_reg_val
8'h08 : o_bus_rdata = w_control_reg_val
8'h0C : o_bus_rdata = {3'b000, i_rx_fifo_level}
8'h10 : o_bus_rdata = o_tx_fifo_wr_data
default: o_bus_rdata = 8'h00
(조합 출력 — 주소가 바뀌면 같은 cycle 에 반영)
```

o_bus_rdata[7:0] → o_bus_rdata

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC — SHEET 5/9

MODULE DETAIL — REGISTER BLOCK

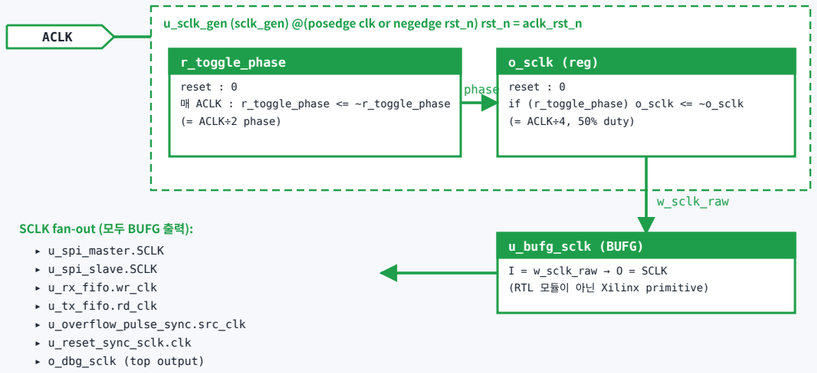
u_register_block (module register_block) · ACLK domain

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…) — DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

CLOCK GENERATION

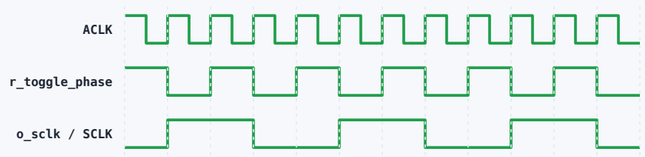
ACLK 100 MHz → SCLK 25 MHz (÷4)



SCLK fan-out (모두 BUFG 출력):

- u_spi_master.SCLK
- u_spi_slave.SCLK
- u_rx_fifo.wr_clk
- u_tx_fifo.rd_clk
- u_overflow_pulse_sync.src_clk
- u_reset_sync_sclk.clk
- o_dbg_sclk (top output)

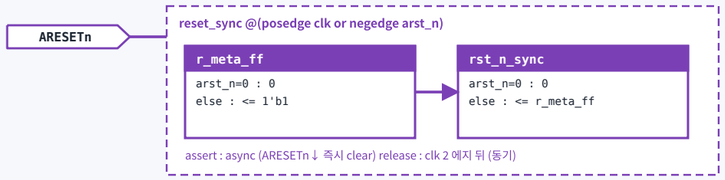
동작 파형 (재구성 · RTL 로직에서 계산한 이상 파형, 시뮬레이션 캡처 아님)



reset 해제 직후부터. SCLK 상승은 ACLK 2번째 에지, 주기 = 4 ACLK = 40 ns

RESET ARCHITECTURE

async assert / sync release · 도메인별 reset_sync



u_reset_sync_aclk (clk = ACLK) → aclk_rst_n

- u_sclk_gen.rst_n (async negedge)
- u_rx_fifo.rd_rst_n / u_tx_fifo.wr_rst_n (async negedge)
- u_overflow_pulse_sync.dst_rst_n (async negedge)
- u_register_block.rst_n (sync, posedge ACLK)
- top r_spi_busy_meta / _sync (sync, posedge ACLK)

u_reset_sync_sclk (clk = SCLK) → sclk_rst_n

- u_spi_master.rst_n / u_spi_slave.rst_n (async negedge)
- u_rx_fifo.wr_rst_n / u_tx_fifo.rd_rst_n (async negedge)
- u_overflow_pulse_sync.src_rst_n (async negedge)

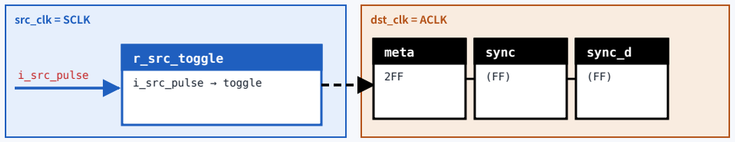
RDC 관련 (구조 관찰 — 도구 검증 아님)

· SCLK 은 aclk_rst_n 동안 low 로 정지 → sclk_rst_n 해제는 SCLK 재시작 뒤 2 SCLK 에지
· SCLK 도메인 FF 는 async reset 사용 (RTL-007 수정 사유). 정적 RDC 분석: 미수행

CDC STRUCTURES

pulse_sync · level 2FF · FIFO pointer 2FF

pulse_sync (u_overflow_pulse_sync) SCLK → ACLK



o_dst_pulse = r_dst_sync ^ r_dst_sync_d (ACLK 1-cycle)

level 2FF (top) busy : SCLK → ACLK ⚠ CDC-02



FIFO pointer 2FF (Gray, async_fifo 내부)



unsynchronized ⚠ CDC-01



CDC INVENTORY — 도메인 교차 신호 전수 (RTL 포트맵/코드 기준, 정적 CDC 도구 검증 아님)

ID	신호 / 경로	방향	구조	RTL 위치	평가 (RTL 정독 기준)
CDC-F1	u_rx_fifo wr ptr (Gray) → r_wr_ptr_gray_meta_r → _rclk	SCLK → ACLK	Gray 코드 + 2FF (data 는 동기화 안 함)	L923–931	구조적으로 표준 Gray-pointer 동기화
CDC-F2	u_rx_fifo rd ptr (Gray) → r_rd_ptr_gray_meta_w → _wclk	ACLK → SCLK	Gray 코드 + 2FF	L885–893	〃
CDC-F3	u_tx_fifo wr ptr (Gray) → …_rclk	ACLK → SCLK	Gray 코드 + 2FF	L923–931	〃 (같은 모듈, ADDR_WIDTH=2)
CDC-F4	u_tx_fifo rd ptr (Gray) → …_wclk	SCLK → ACLK	Gray 코드 + 2FF	L885–893	〃
CDC-P1	w_rx_overflow_pulse → u_overflow_pulse_sync → w_rx_overflow_pulse_sync	SCLK → ACLK	toggle + 2FF + XOR edge detect (pulse_sync)	L1183–1206 / top L193–199	1-cycle pulse 전달 구조. 연속 펄스 간격 제한(src ≥ 2 dst 주기 등)은 미분석
CDC-02	w_spi_busy_master ‖ w_spi_busy_slave → r_spi_busy_meta → _sync	SCLK → ACLK	조합 OR(여러 상태 FF 디코드) 후 2FF level sync	top L207–217	조합 출력이 직접 2FF 에 입력 — glitch 가능성 (미수정, 정적 CDC 도구 미수행)
CDC-01	u_rx_fifo.o_full (r_full, SCLK) → register_block.i_rx_fifo_full → STATUS[1]	SCLK → ACLK	동기화 없음 (직결)	top L235 / reg L1124	단일 bit level 이 동기기 없이 사용 — 미수정 열린 항목
CDC-D1	FIFO mem[] data (u_rx_fifo: SCLK write → ACLK read, u_tx_fifo: ACLK write → SCLK read)	양방향	dual-clock RAM + 포인터 순서 보장	FIFO L880 / L918	데이터 비트 동기화 없음(정상). 포인터 프로토콜로 보호됨
(참고)	ARESETn (외부) → reset_sync ×2	async → ACLK, SCLK	async assert / sync release	L1228–1250	외부 reset 은 두 도메인 reset_sync 로 입력. 입력 타이밍 제약: 미확인
(참고)	i_bus_* → register_block	외부 → ACLK	동기화 로직 없음 (ACLK 동기 입력 가정)	reg block	입력 클럭 관계는 SDC 로 정의되어야 함: 미확인
(참고)	o_dbg_cs_n / mosi / miso (SCLK 도메인) → 외부 핀	SCLK → 외부	직결 (debug 출력)	top L130–133	외부 샘플링 도메인 없음. 디버그 용도

※ CDC-F1~F4 는 구조 확인(RTL). 위 어느 항목도 정적 CDC/RDC 도구, 합성, STA 로 검증된 결과가 아님(미수행). L### = spi_bridge_top.v 줄 번호 (사용자 이름 기준).

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC

SHEET 6/9

CLOCK / RESET / CDC ARCHITECTURE

sclk_gen · reset_sync · pulse_sync · FIFO pointer sync · 인벤토리

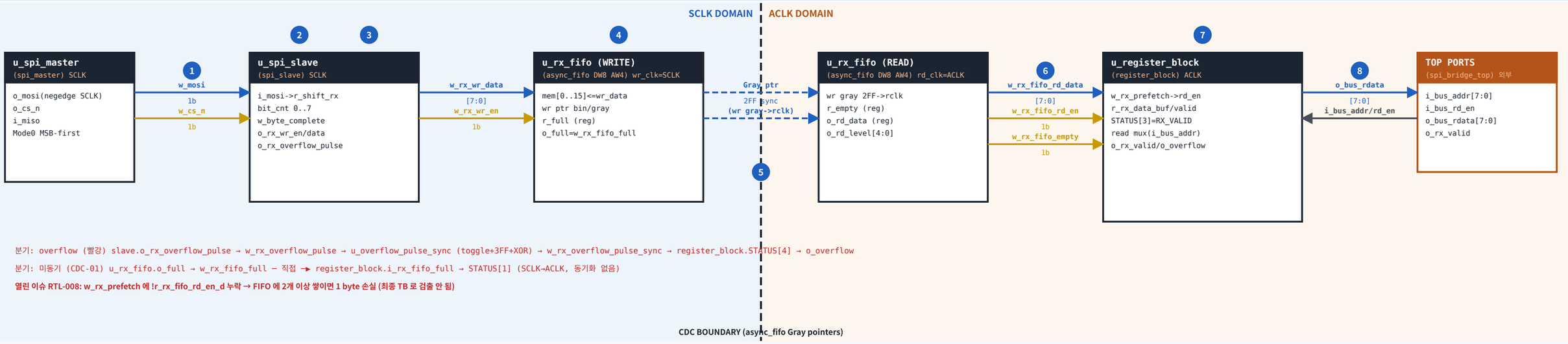
SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…)

DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

SIGNAL FLOW — RX PATH

SPI master → SPI slave → u_rx_fifo (CDC) → u_register_block → i_bus_*/o_bus_rdata



분기: overflow (빨강) slave.o_rx_overflow_pulse → w_rx_overflow_pulse → u_overflow_pulse_sync (toggle+3FF+XOR) → w_rx_overflow_pulse_sync → register_block.STATUS[4] → o_overflow

분기: 미동기 (CDC-01) u_rx_fifo.o_full → w_rx_fifo_full — 직접 → register_block.i_rx_fifo_full → STATUS[1] (SCLK→ACLK, 동기화 없음)

열린 이슈 RTL-008: w_rx_prefetch 에 !r_rx_fifo_rd_en_d 누락 → FIFO 에 2개 이상 쌓이면 1 byte 손실 (최종 TB 로 검출 안 됨)

#	동작 (RTL 기준으로 수정된 단계)	신호	RTL 위치 (spi_bridge_top.v 줄)
1	master 가 w_mosi 로 직렬 송신 (negedge SCLK 에 launch)	u_spi_master.o_mosi → w_mosi	L420 (o_mosi negedge 블록)
2	slave 가 SCLK posedge 에 샘플, MSB-first 시프트	u_spi_slave.i_mosi → r_shift_rx	L633–646
3	8 bit 완료 판정 → 병렬 바이트 확정	w_byte_complete	L572–574
4	RX FIFO write (SCLK 도메인) mem[wr_ptr] <= wr_data	o_rx_wr_en/data → u_rx_fifo.i_wr_*	L771 · FIFO L880
5	CDC: wr Gray 포인터를 rclk(ACLK) 로 2FF 동기 r_wr_ptr_gray_meta_r → _rclk	r_wr_ptr_gray_rclk	L923–931
6	prefetch → FIFO read (ACLK), registered read data r_empty 갱신	w_rx_fifo_rd_en / rd_data / empty	L1032,1039 · FIFO L912,918
7	rx_data_buf_valid / r_rx_data_buf 갱신 (RTL-008 위치)	r_rx_data_buf, rx_data_buf_valid	L1043–1058
8	i_bus_rd_en · addr 0x00 → o_bus_rdata 반환 STATUS(0x04) 로 RX_VALID/FULL/EMPTY 확인	o_bus_rdata[7:0]	L1120–1131

※ 사전 가이드의 “RX FIFO Write → CDC → Read” 순서는 RTL 과 일치. 단 master/slave 가 모두 top 내부에 있으며 외부 SPI 핀은 없음(o_dbg_* 는 관찰용 출력).

※ 확인 수준: 정적 RTL 검토. 시뮬레이션/CDC 도구 검증 결과 아님.

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC — SHEET 7/9

RX DATAFLOW

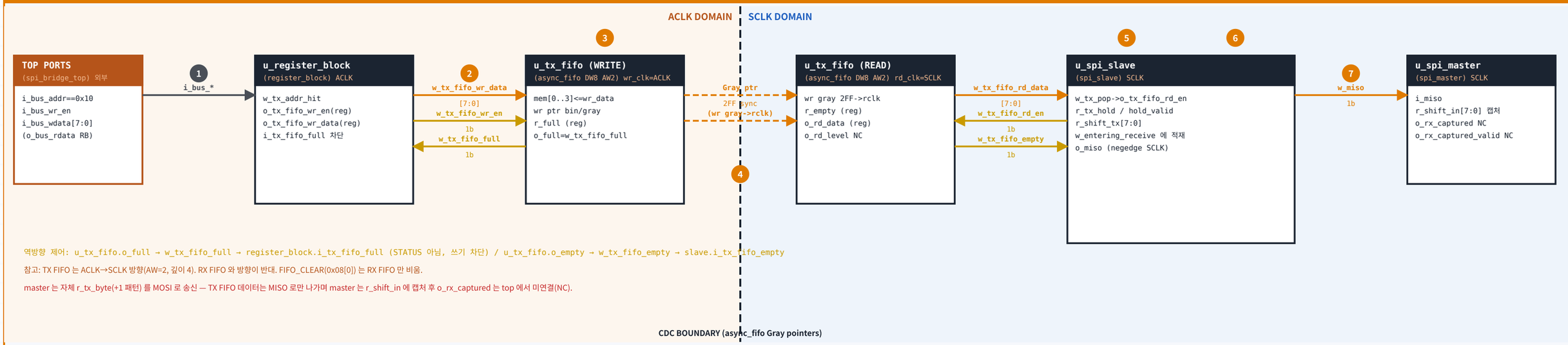
master → slave → u_rx_fifo → register_block → bus

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…) — DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

SIGNAL FLOW — TX PATH

i_bus_wr_en/addr 0x10 → u_register_block → u_tx_fifo (CDC) → u_spi_slave → w_miso → u_spi_master



역방향 제어: u_tx_fifo.o_full → w_tx_fifo_full → register_block.i_tx_fifo_full (STATUS 아님, 쓰기 차단) / u_tx_fifo.o_empty → w_tx_fifo_empty → slave.i_tx_fifo_empty

참고: TX FIFO 는 ACLK→SCLK 방향(AW=2, 깊이 4). RX FIFO 와 방향이 반대. FIFO_CLEAR(0x08[0]) 는 RX FIFO 만 비움.

master 는 자체 r_tx_byte(+1 패턴) 를 MOSI 로 송신 — TX FIFO 데이터는 MISO 로만 나가며 master 는 r_shift_in 에 캡처 후 o_rx_captured 는 top 에서 미연결(NC).

#	동작 (RTL 기준으로 수정된 단계)	신호	RTL 위치 (spi_bridge_top.v 줄)
1	bus write: i_bus_wr_en=1, i_bus_addr==0x10, i_bus_wdata	i_bus_addr/wr_en/wdata → w_tx_addr_hit	L1028
2	register_block 이 ACLK posedge 에 등록 wr_en <= wr_en & hit & !full	o_tx_fifo_wr_en/data → w_tx_fifo_wr_*	L1111–1113
3	TX FIFO 저장 (ACLK 도메인) mem[wr_ptr] <= wr_data	u_tx_fifo.i_wr_en/i_wr_data	FIFO L880
4	CDC: wr Gray 포인터 → SCLK 2FF → r_empty 해제	r_wr_ptr_gray_rclk (rclk=SCLK)	L923–931 · L912
5	slave 가 pop: w_tx_pop = !empty & !hold_valid & !pop_d registered 읽기 → r_tx_hold	w_tx_fifo_rd_en/rd_data, r_tx_hold	L660–665 · L701–703
6	w_entering_receive 에서 r_shift_tx <= r_tx_hold (hold 無 → 8'h00), hold_valid 소비	r_shift_tx, w_tx_consume	L727–735 · L708
7	o_miso <= r_shift_tx[7] (negedge SCLK) → w_miso master i_miso 에서 r_shift_in 캡처 (o_rx_captured NC)	o_miso → w_miso → i_miso	L755–761 · L441

※ 사전 가이드의 “SPI Slave 의 TX 데이터 로딩 → Shift Register” 사이에 hold 레지스터(r_tx_hold)와 pop 지연(r_tx_pop_d) 단계가 실제로 존재 (RTL-006 수정 결과).

※ 확인 수준: 정적 RTL 검토. 시뮬레이션/CDC 도구 검증 결과 아님.

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC SHEET 8/9

TX DATAFLOW

bus → register_block → u_tx_fifo → u_spi_slave → w_miso → u_spi_master

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…) DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기

STEP 6 — 검증 체크리스트 · 계층 · 미확인

도면 ↔ 연결표(64행) 대조 · 정적 RTL 검토 기준 (도구 검증 아님)

A. 자동 대조 (Sheet A 의 텍스트에 이름이 존재하는지 — 이름 수준 확인, 배선 위상은 사람이 검토)

Top 포트 13개 이름	13/13 OK
인스턴스 10개	10/10 OK
top 내부 net 이름	34/34 OK
인스턴스 포트 이름	57/57 OK

B. STEP 6 항목

항목	판정	근거
모든 실제 Module Instance 포함	충족	top 10개 전부(Sheet A, 자동 대조 10/10). 모듈 선언 8개 중 인스턴스되지 않은 모듈 없음
모든 Top-Level Port 포함	충족	13/13 (입력 6, 출력 7)
모든 Module Port Connection 연결	충족(NC 3 명시)	연결표 64행. 미연결: master.o_rx_captured[7:0]/_valid, tx_fifo.o_rd_level
Signal Width 정확	부분 확인	RTL 선언에서 파싱, 폭 표기는 wire 라벨. 파서 오류 1건(async_fifo 가짜 포트 g) 제외
RX/TX 방향 정확	충족	RX: SCLK→ACLK(16×8), TX: ACLK→SCLK(4×8). RTL 인스턴스 포트 방향으로 확인
Clock Domain 구분 정확	충족	SCLK=ACLK÷4(sclk_gen+BUFG). FIFO/pulse_sync 는 양 도메인에 걸침
Reset 연결 정확	충족	reset_sync ×2, 도메인별 분배. register_block·busy 2FF 는 sync reset
CDC 구조 일치	충족(구조)	Gray 2FF×2, toggle pulse_sync, busy OR+2FF. CDC-01/02 열린 항목을 도면에 표시. 정적 CDC 도구: 미수행
FIFO Full/Empty 신호 정확	충족	registered r_full/r_empty, w_rx_fifo_full/empty, w_tx_fifo_full/empty
Register Map 정확	충족	0x00 RX_DATA, 0x04 STATUS, 0x08 CONTROL, 0x0C FIFO_LEVEL, 0x10 TX_DATA (RTL localparam)
미구현 기능 미포함	충족	CPU/AXI/IRQ 출력/외부 SPI 핀/별도 clock divider 모듈/Status&Interrupt 블록 없음
신호 누락/오연결 없음	부분 확인	이름 수준 대조 통과. 배선 하나하나의 시각 검토는 사람 몫 → 미확인으로 남김

C. 미확인 / 한계 (추측으로 그리지 않음)

· i_bus_* 입력 타이밍 제약(set_input_delay)·ARESETn 외부 제약: RTL 에 없음 → 미확인
· 정적 CDC/RDC, lint, 합성, STA, LEC: 미수행 — 본 도면은 코드 구조 확인(RTL review)이며 tool verification 이 아님
· irq_enable(0x08[2]): RTL 에서 소비처 없음 (출력 없음) · FIFO_CLEAR 는 RX FIFO 만 drain
· RTL-008(RX prefetch double-pop), CDC-01(rx_full 미동기 → STATUS[1]), CDC-02(busy OR 후 2FF): 열린 이슈, 도면에 표시

D. Module Hierarchy

Module	Instance	Parent	Clock	Reset	Function
spi_bridge_top	(top)	—	ACLK (입력), SCLK (내부 생성)	ARESETn (async 입력)	최상위: 클럭/리셋 생성, 내부 SPI master↔slave 직결, 두 개의 async FIFO, 레지스터 블록, busy 2FF 동기화, debug 출력
reset_sync	u_reset_sync_aclk	spi_bridge_top	ACLK	ARESETn → aclk_rst_n	async assert / sync release (2FF), ACLK 도메인 리셋
reset_sync	u_reset_sync_sclk	spi_bridge_top	SCLK	ARESETn → sclk_rst_n	async assert / sync release (2FF), SCLK 도메인 리셋
sclk_gen	u_sclk_gen	spi_bridge_top	ACLK	aclk_rst_n (async)	ACLK÷4 분주 (r_toggle_phase + o_sclk), 25 MHz
BUFG (Xilinx prim…	u_bufg_sclk	spi_bridge_top	—	—	w_sclk_raw → SCLK 전역 클럭 버퍼 (RTL 모듈 아님, 라이브러리 primitive)
spi_master	u_spi_master	spi_bridge_top	SCLK	sclk_rst_n (async)	내부 SPI master: 4-state FSM, 8-bit 프레임 송신 (패턴 r_tx_byte++), MISO 수신
spi_slave	u_spi_slave	spi_bridge_top	SCLK	sclk_rst_n (async)	SPI slave: 4-state FSM, RX shift, TX hold/shift, FIFO 인터페이스
async_fifo	u_rx_fifo	spi_bridge_top	wr: SCLK / rd: ACLK	wr: sclk_rst_n / rd: aclk_rst_n	RX FIFO, DATA_WIDTH=8, ADDR_WIDTH=4 (16×8), SCLK→ACLK
async_fifo	u_tx_fifo	spi_bridge_top	wr: ACLK / rd: SCLK	wr: aclk_rst_n / rd: sclk_rst_n	TX FIFO, DATA_WIDTH=8, ADDR_WIDTH=2 (4×8), ACLK→SCLK
pulse_sync	u_overflow_pulse_sync	spi_bridge_top	src: SCLK / dst: ACLK	src: sclk_rst_n / dst: aclk_rst_n	RX overflow pulse SCLK→ACLK (toggle + 2FF + XOR)
register_block	u_register_block	spi_bridge_top	ACLK	aclk_rst_n (sync)	레지스터 버스(0x00..0x10), RX prefetch 버퍼, drain, overflow sticky, TX write

SPI SLAVE CDC BRIDGE — RTL-ACCURATE SCHEMATIC SHEET 9/9

VERIFICATION · HIERARCHY

STEP 6 checklist · 미확인 항목 · module hierarchy

SOURCE spi_bridge_top.v (1,293 lines, SHA-256 f7cfaff9…) DATE 2026-10-21

모든 연결은 RTL 인스턴스 포트맵에서 추출·검증 / 확인 불가 항목은 "미확인" 표기